EC-360
SOFTWARE ENGINEERING

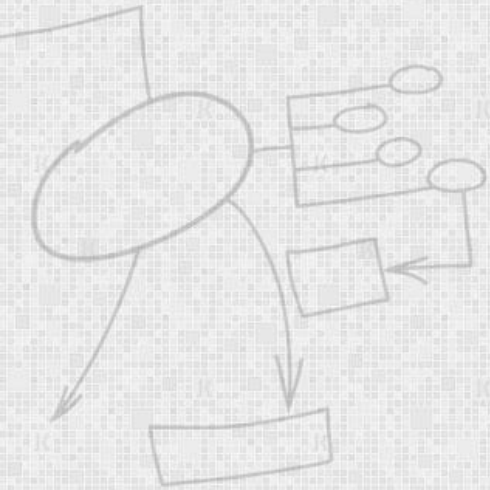
Lecture 9

Unified Modeling Language (UML) for Architecture and Design

By
Dr Wasi Haider Butt

Agenda

- Unified Modeling Language
- Architecture using UML
- Low level design using UML



What is UML?

UML: **U**nified **M**odeling **L**anguage

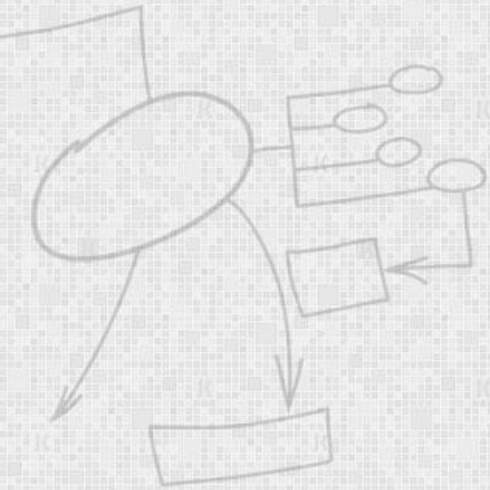
- An industry-standard graphical language for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modeling.

- The UML uses mostly graphical notations to express the OO analysis and design of software projects.

- Simplifies the complex process of software design

Why UML for Modeling?

- Uses graphical notation to communicate more clearly than natural language (imprecise) and code(too detailed).
- Makes it easier for programmers and software architects to communicate.
- Helps acquire an overall view of a system.

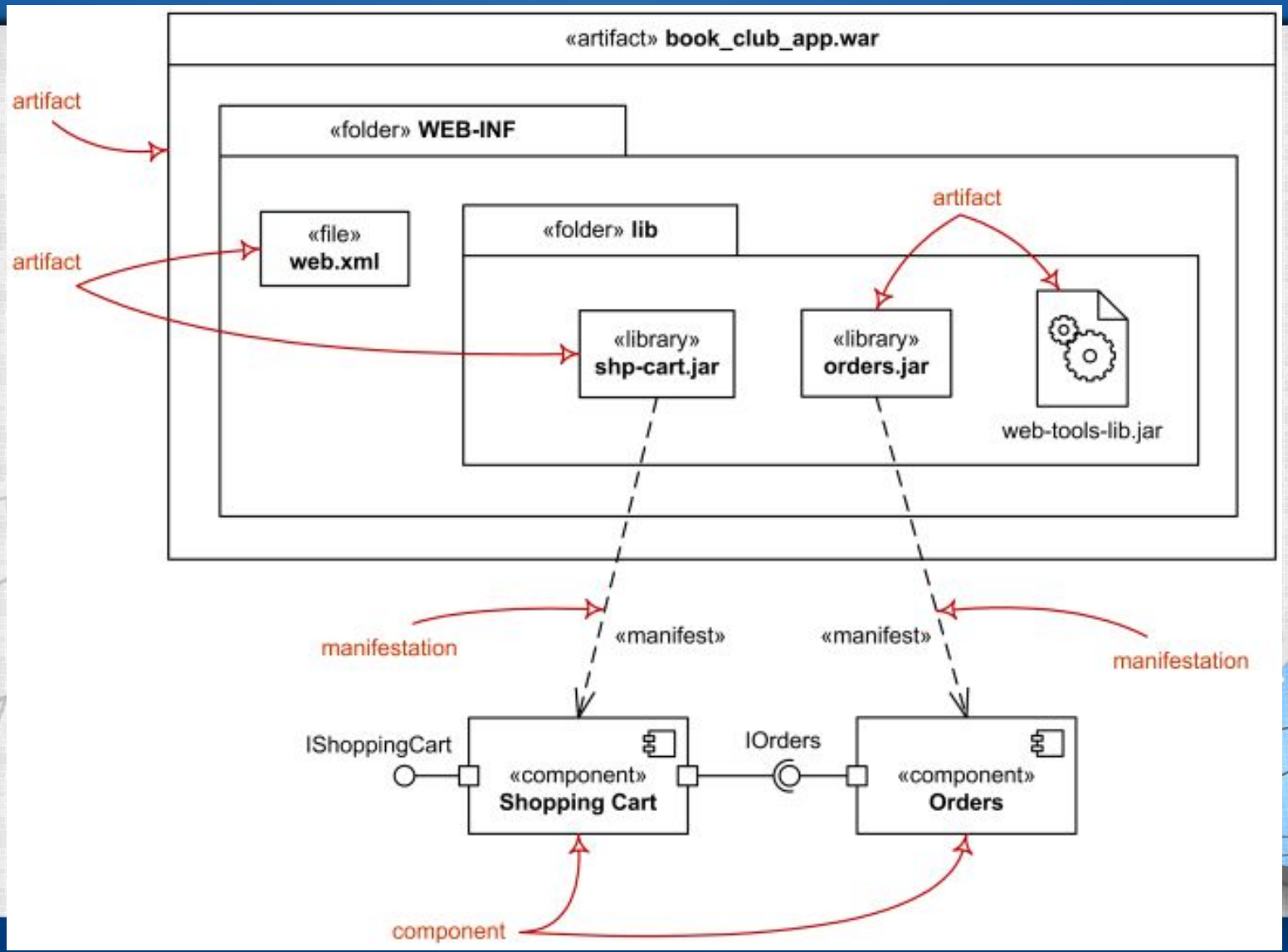


Deployment Diagrams

- Deployment diagram is a structure diagram which shows architecture of the system as deployment (distribution) of software artefacts to deployment targets.
- Some common types of deployment diagrams are:
  - Implementation (manifestation) of components by artefacts
  - Specification level deployment diagram,
  - Instance level deployment diagram,
  - Network architecture of the system.

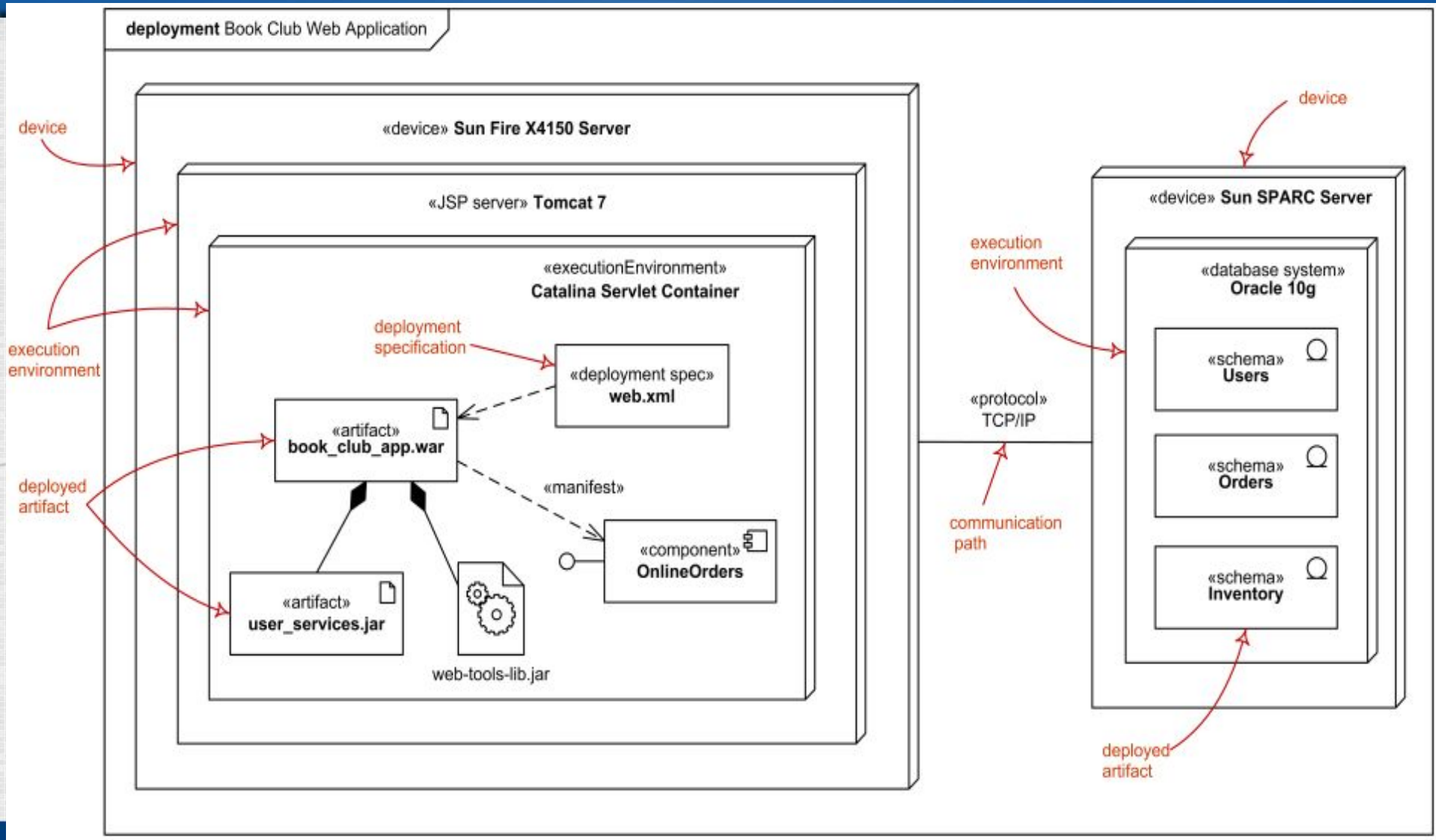
Manifestation (implementation**) of Components by Artifacts**

Used to show manifestation (implementation) of components by artifacts and internal structure of artifacts



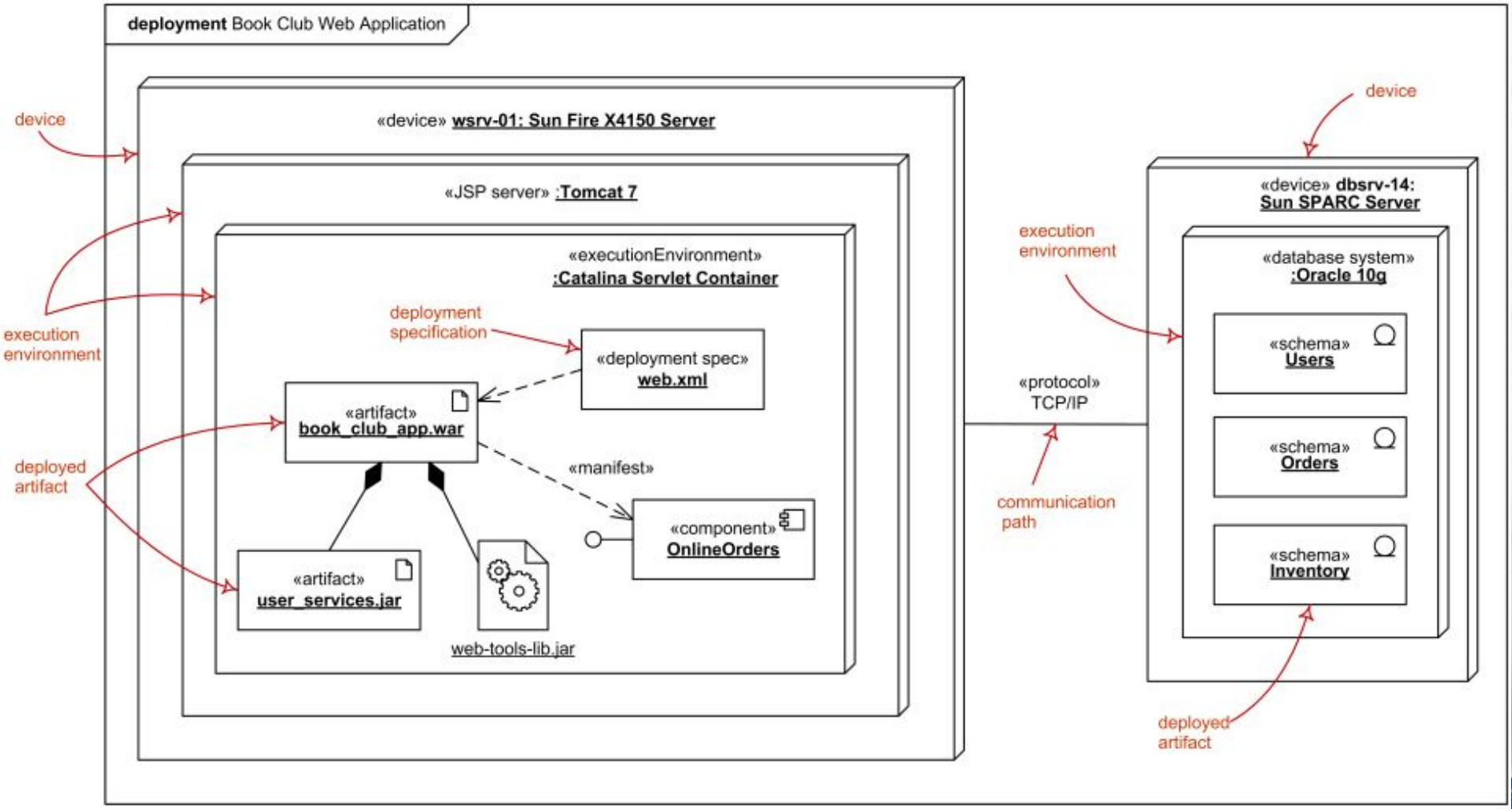
Specification (Type) Level Deployment Diagram

Shows some overview of deployment of artifacts to deployment targets, without referencing specific instances of artifacts or nodes



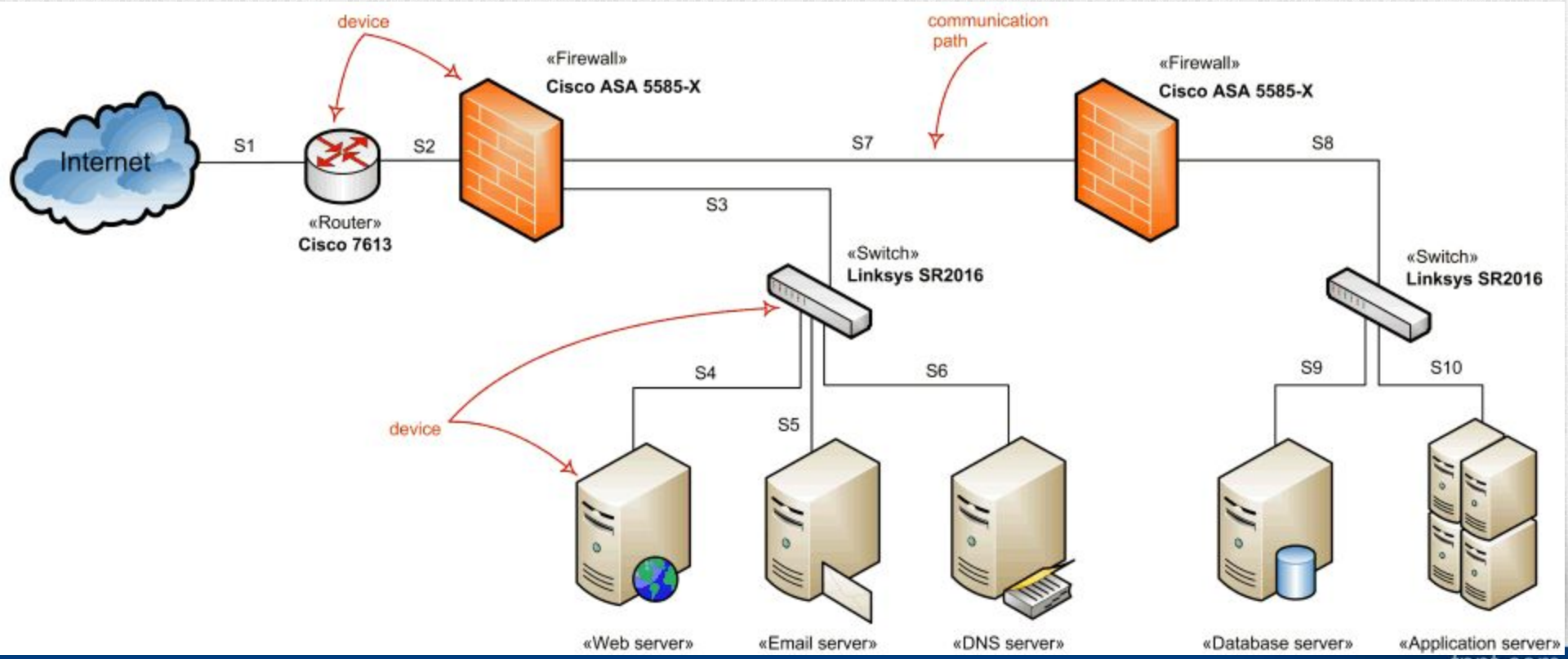
Instance Level Deployment Diagram

Shows deployment of instances of artifacts to specific instances of deployment targets.
E.g, Web application is deployed to the application server **wsrv-01** and several database schemas - to the database server **dbsrv-14**



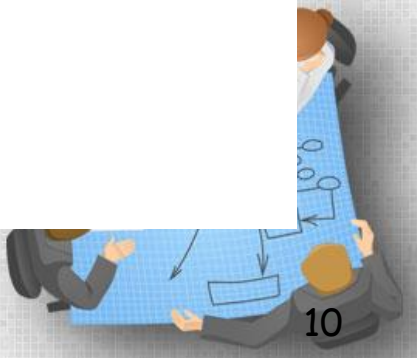
Network architecture diagrams

- **UML** standard has no separate kind of diagrams to describe **network architecture** and provides no specific elements related to the networking
- Deployment diagrams could be used for this purpose usually with some extra networking stereotypes.
- Network architecture diagram will usually show networking nodes and communication paths between them.

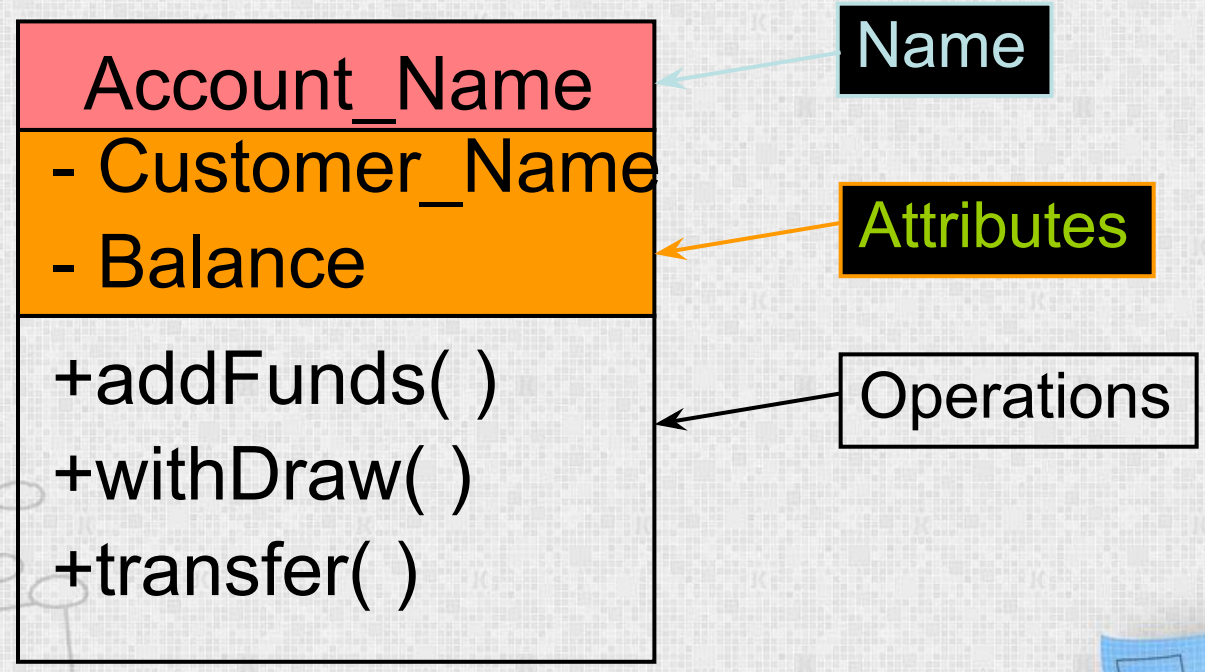


Class diagram

- A static structure diagram which describes structure of a system at the level of classifiers (classes, interfaces, etc.).
- It shows some classifiers of the system, subsystem or component, different relationships between classifiers, their attributes, operations and constraints.



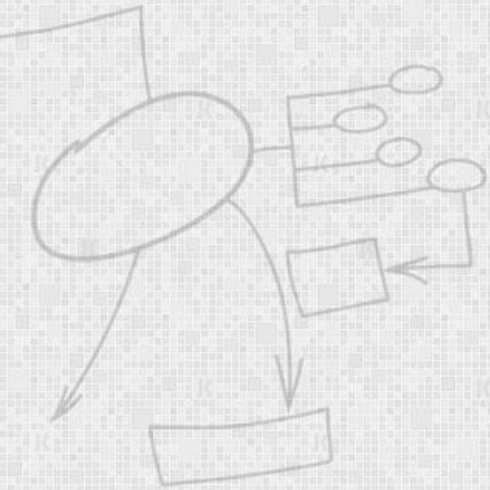
Class diagram



Classes

Characteristics represented by feature may be of the classifier's instances considered individually (**not static**) or of the classifier itself (**static**).
Static features are **underlined** - but only the names.

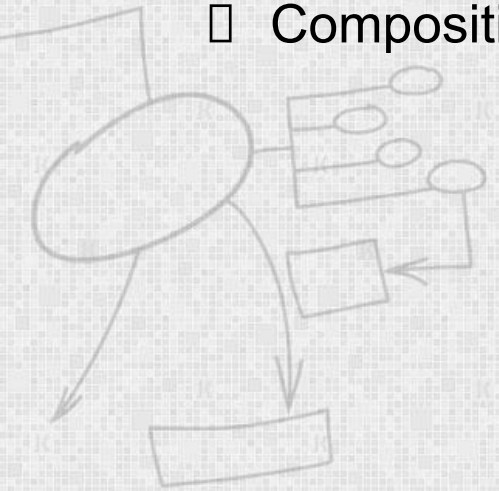
Objects of a class must contain values for each attribute that is a member of that class, in accordance with the characteristics of the attribute, for example its type and multiplicity.



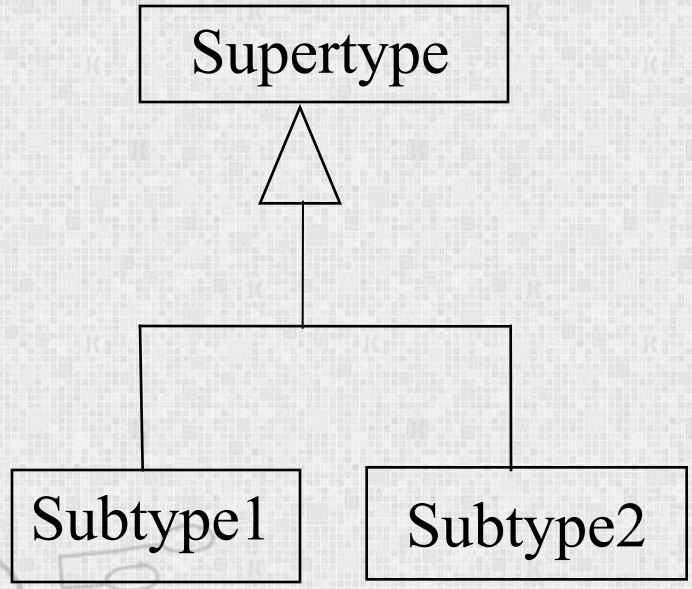
SearchService
- config: Configuration - engine: SearchEngine
+ search(query: SearchRequest): SearchResult - createEngine(): SearchEngine

OO Relationships

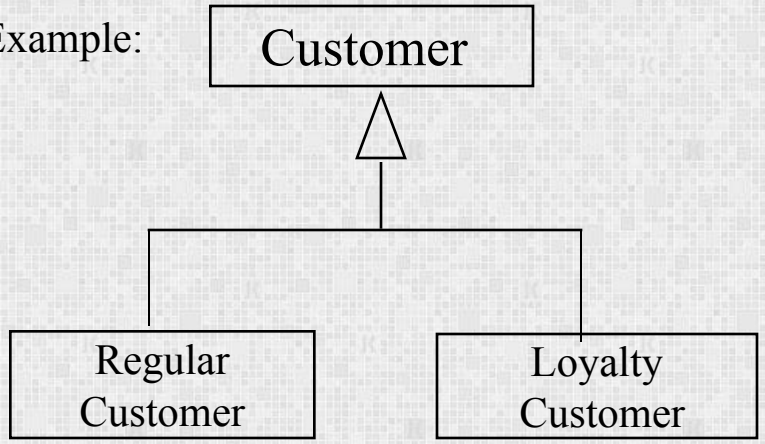
- There are two kinds of Relationships
  - Generalization (parent-child relationship)
  - Association (student enrolls in course)
- Associations can be further classified as
  - Aggregation
  - Composition



OO Relationships: Generalization



Example:



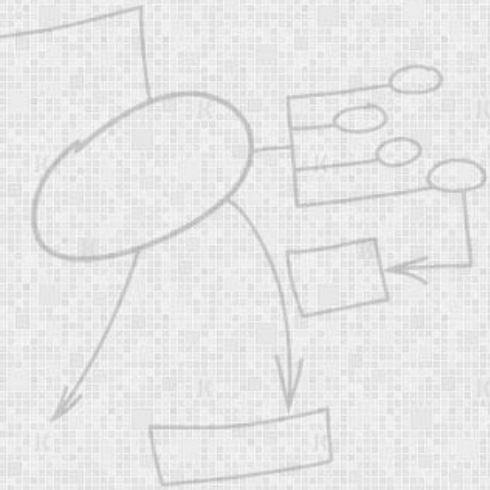
-Inheritance is a required feature of object orientation

-Generalization expresses a parent/child relationship among related classes.

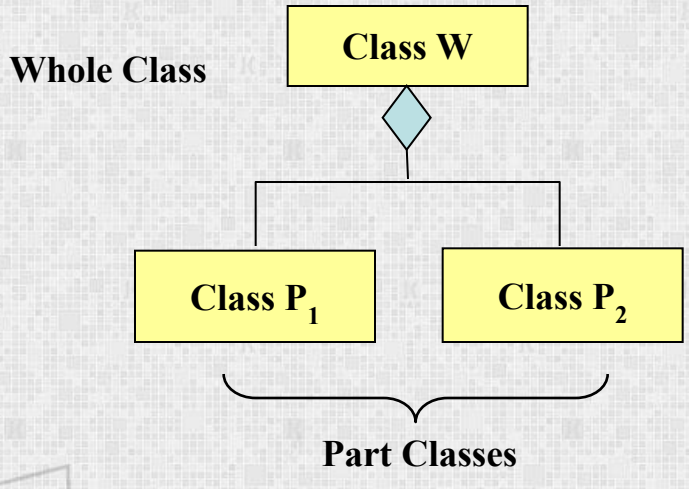
-Used for abstracting details in several layers

OO Relationships: Association

- Represent relationship between instances of classes
  - Student enrolls in a course
  - Courses have students
  - Courses have exams
  - Etc.



OO Relationships: Composition

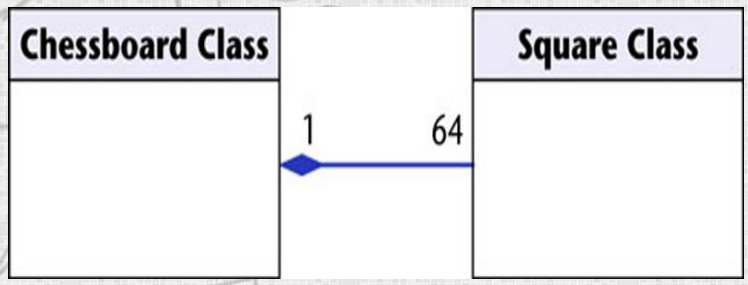


Composition

The part class cannot exist independently.

House has rooms. If house is deleted, rooms are also gone

Example




```
public class Engine
{
...
}

public class Car
{
   Engine e = new Engine();
   .......
}
```

OO Relationships: Aggregation

```
public class Address
{
 . . .
}

public class Person
{
    private Address address;
    public Person(Address address)
    {
       this.address = address;
    }
    . . .
}
```

Person would then be used as follows:

```
Address address = new Address();
Person person = new Person(address);
```

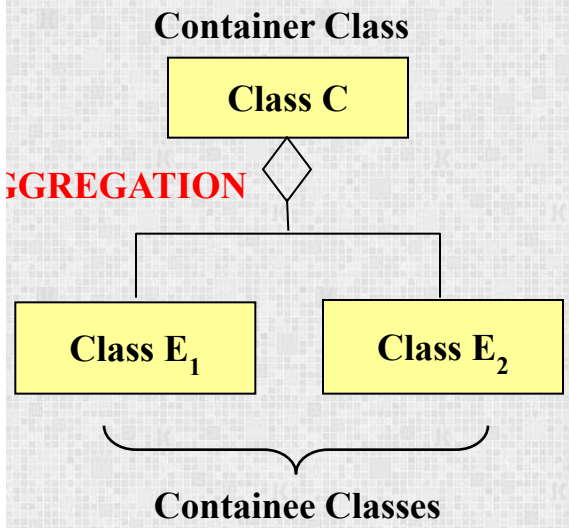
or

```
Person person = new Person( new Address() );
```

Aggregation:

Part class can exist independently

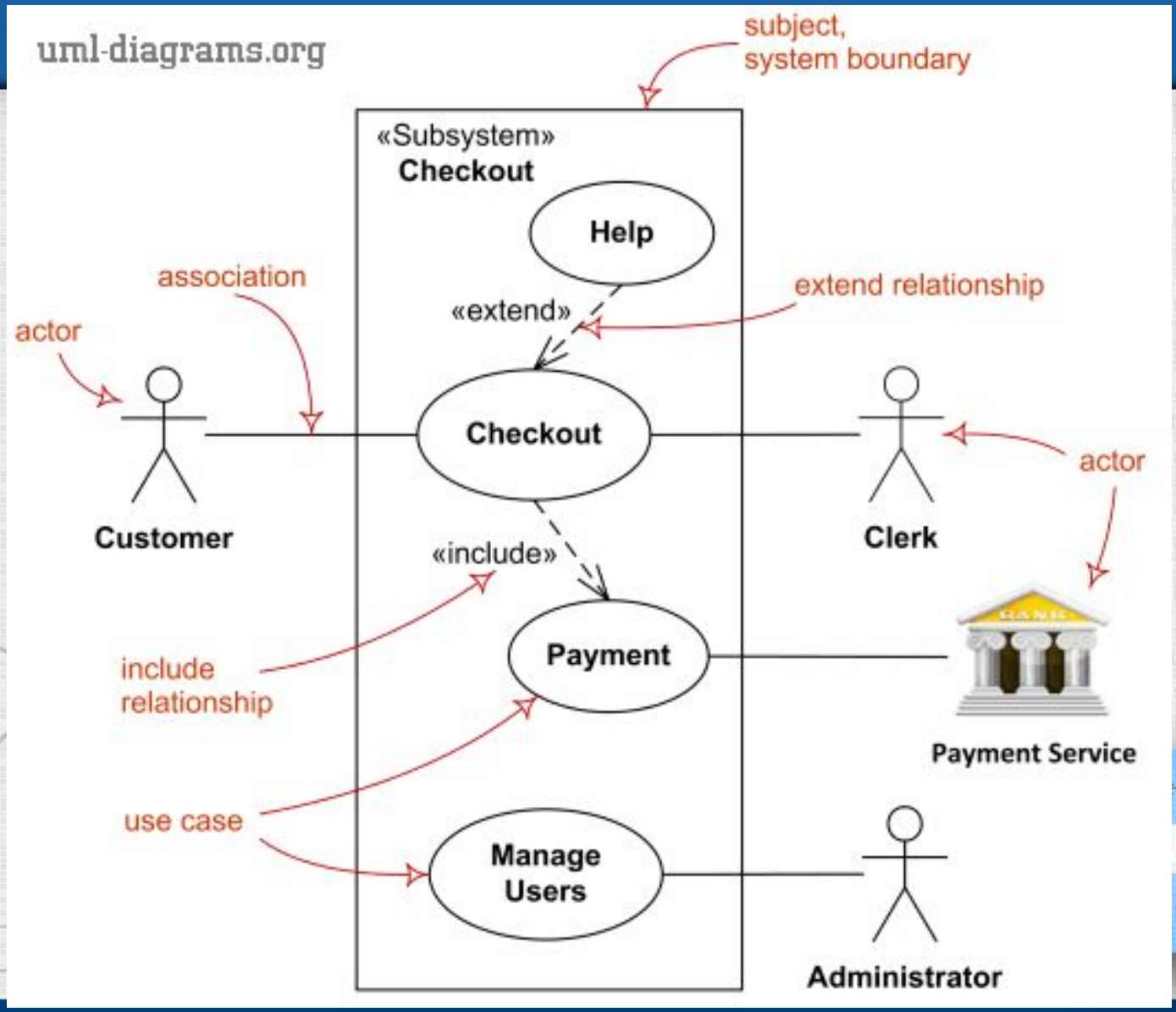
Class is composed of students. If class is deleted, students still exist



Properties

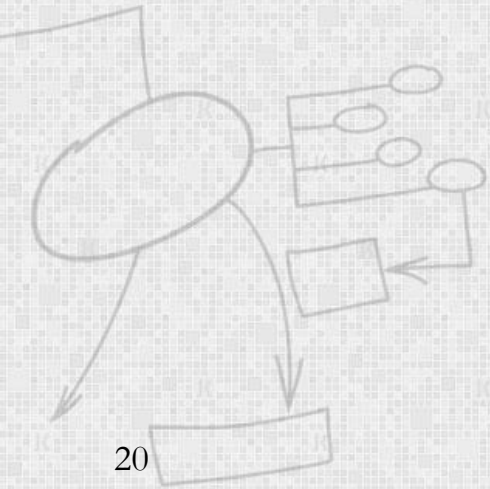
```
class TimePeriod
{
    private double seconds;

    public double Hours
    {
        get { return seconds / 3600; }
        set { seconds = value * 3600; }
    }
}


class Program
{
    static void Main()
    {
        TimePeriod t = new TimePeriod();

        // Assigning the Hours property causes the 'set' accessor to be called.
        t.Hours = 24;

        // Evaluating the Hours property causes the 'get' accessor to be called.
        System.Console.WriteLine("Time in hours: " + t.Hours);
    }
}
// Output: Time in hours: 24
```

System Use Case Diagram



Use Case Descriptions

- ***Title*** – descriptive name, matches name in use case diagram
- ***Primary actor*** – usually a user role
- ***Precondition*** – conditions that must be satisfied in order to execute the use case



Use Case Description

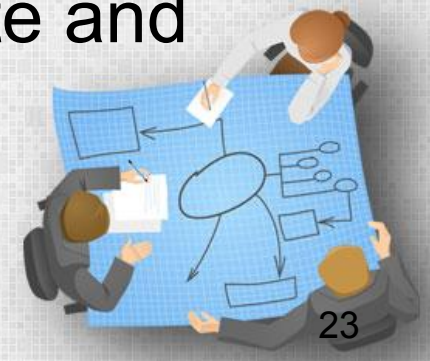
- ***Postcondition*** – outputs that can be expected if the service succeeds
- ***Main success scenario*** – description of sequence of interactions between actor and use case during the use case execution
- ***Alternate scenario*** – Paths other than main success scenario
- ***Extensions*** – detailed description of how errors are dealt with

Validate PIN Use Case - 1

- Name: Validate PIN
- Summary : System validates customer PIN
- Dependency: none
- Actors: ATM Customer
- Preconditions: ATM is idle, displaying a Welcome message.

Validate PIN Use Case - 2

- Flow of Events: Basic Path
  - 1. Customer inserts the ATM card into the Card Reader
  - 2. If the system recognizes the card, it reads the card number
  - 3. System prompt customer for PIN number
  - 4. Customer enters PIN
  - 5. System checks the expiration date and whether the card is lost or stolen



Validate PIN Use Case - 3

- Flow of Events (cont.):
  - 6. If card is valid, the system then checks whether the user-entered PIN matches the card PIN maintained by the system
  - 7. If PIN numbers match, the system checks what accounts are accessible with the ATM card
  - 8. System displays customer accounts and prompts customer for transaction type: Withdrawal, Query, or Transfer

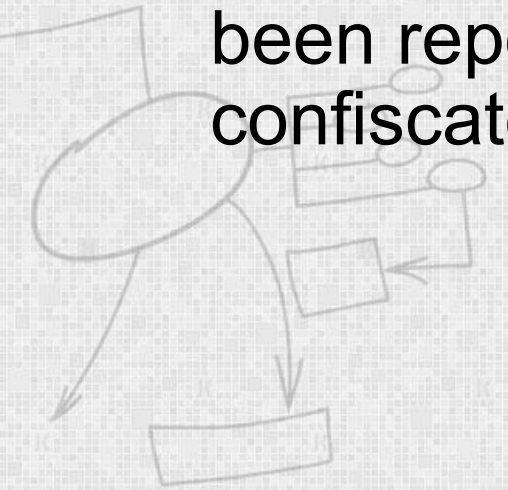
Validate PIN Use Case - 4

- Alternatives:

  2 If the system does not recognize the card, the card is ejected

  5a If the system determines that the card date has expired, the card is confiscated

  5b If the system determines that the card has been reported lost or stolen, the card is confiscated

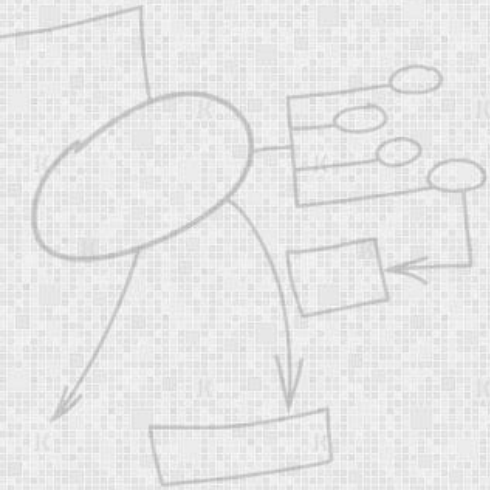


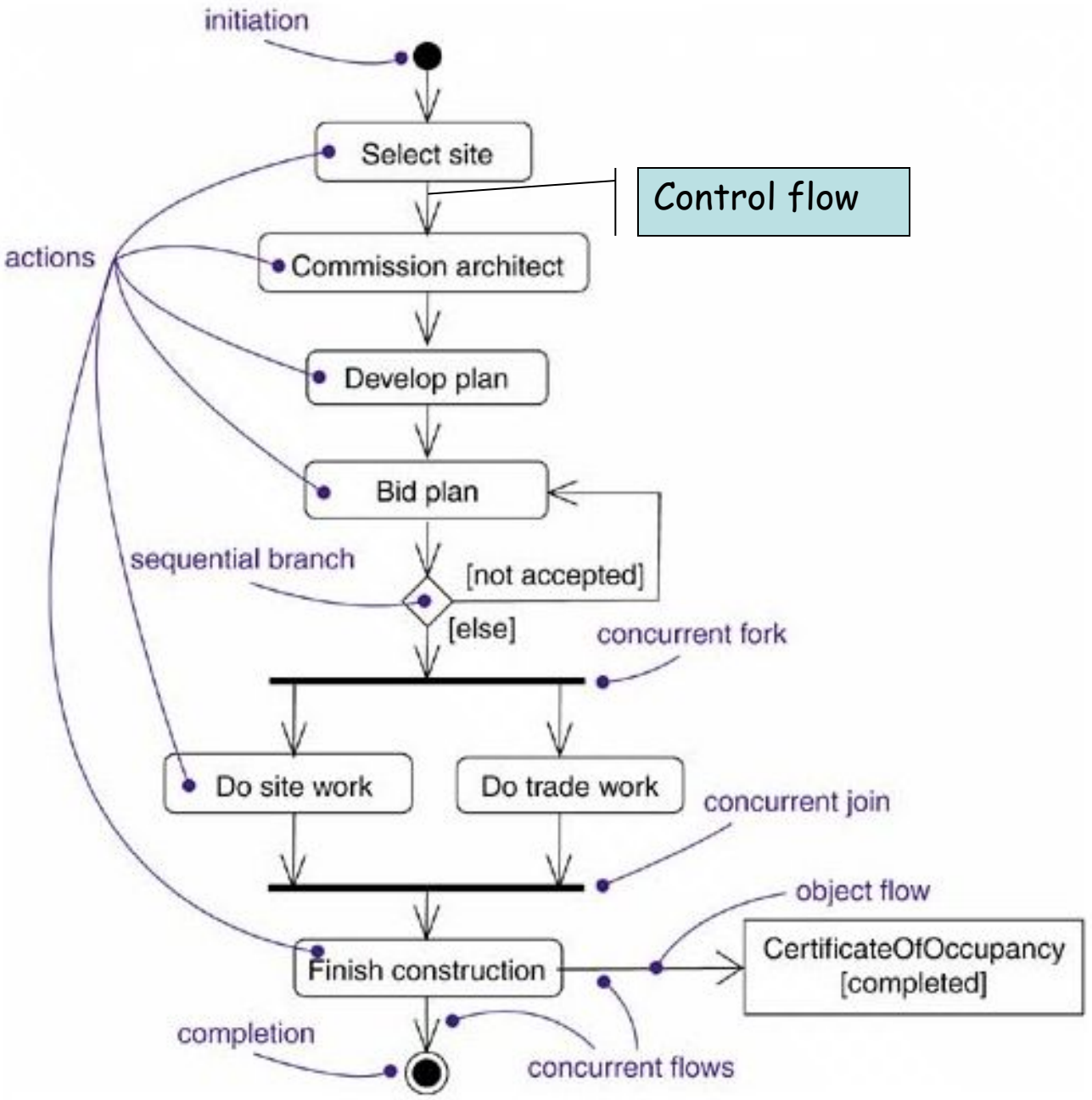
Validate PIN Use Case - 5

- Alternatives (cont.):
  - If the customer-entered PIN does not match the PIN number for this card, the system re-prompts for PIN
  - If the customer enter the incorrect PIN three times, the system confiscates the card
  - If the customer enters Cancel, the system cancels the transaction and ejects the card
- Postcondition: Customer PIN has been checked and response given

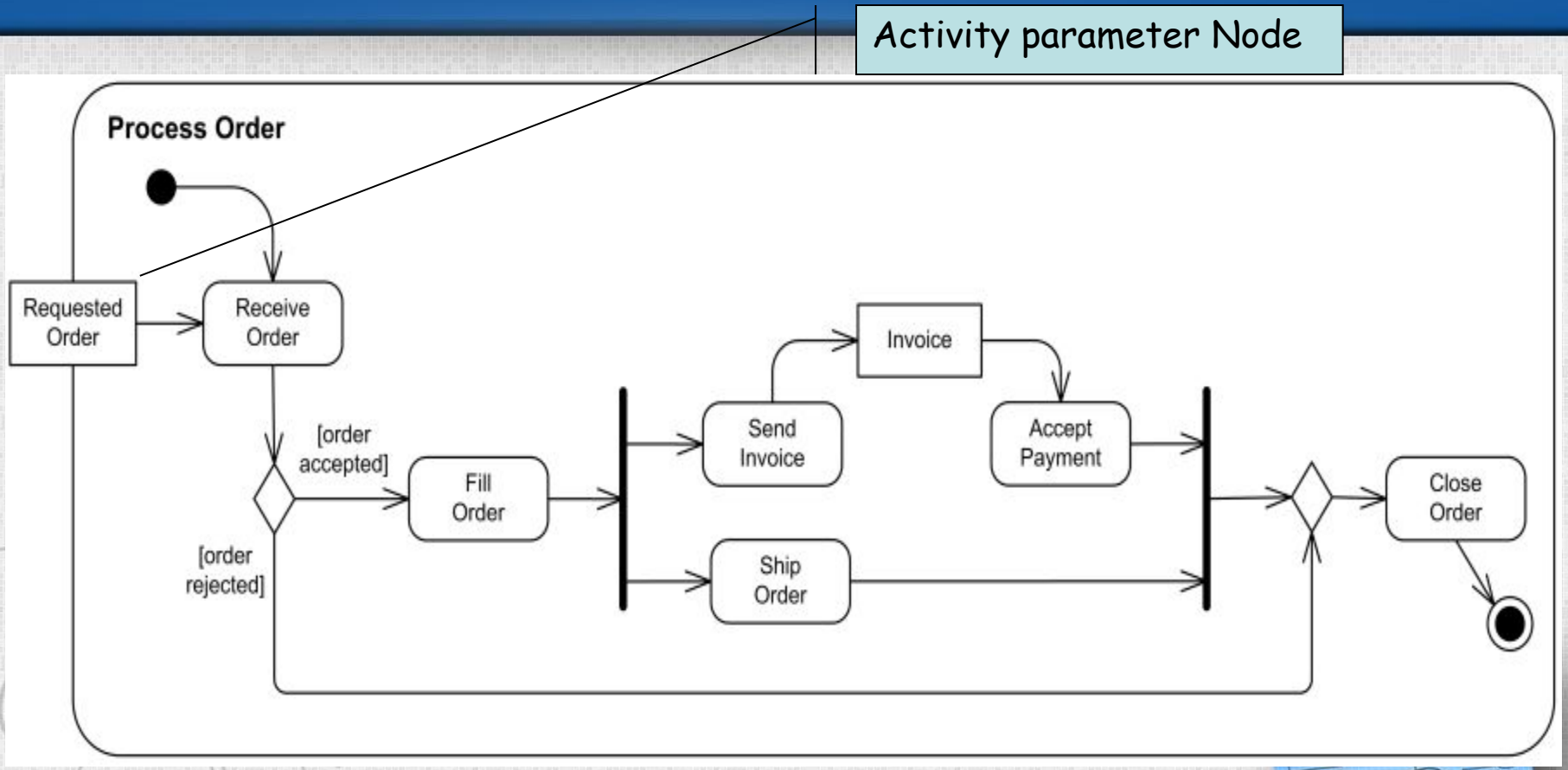
Activity Diagrams

- Activity diagram is UML behaviour diagram which shows flow of control or object flow with emphasis on the sequence and conditions of the flow.
- The following nodes and edges are typically drawn on UML activity diagrams:
  - activity, partition, action, object, control

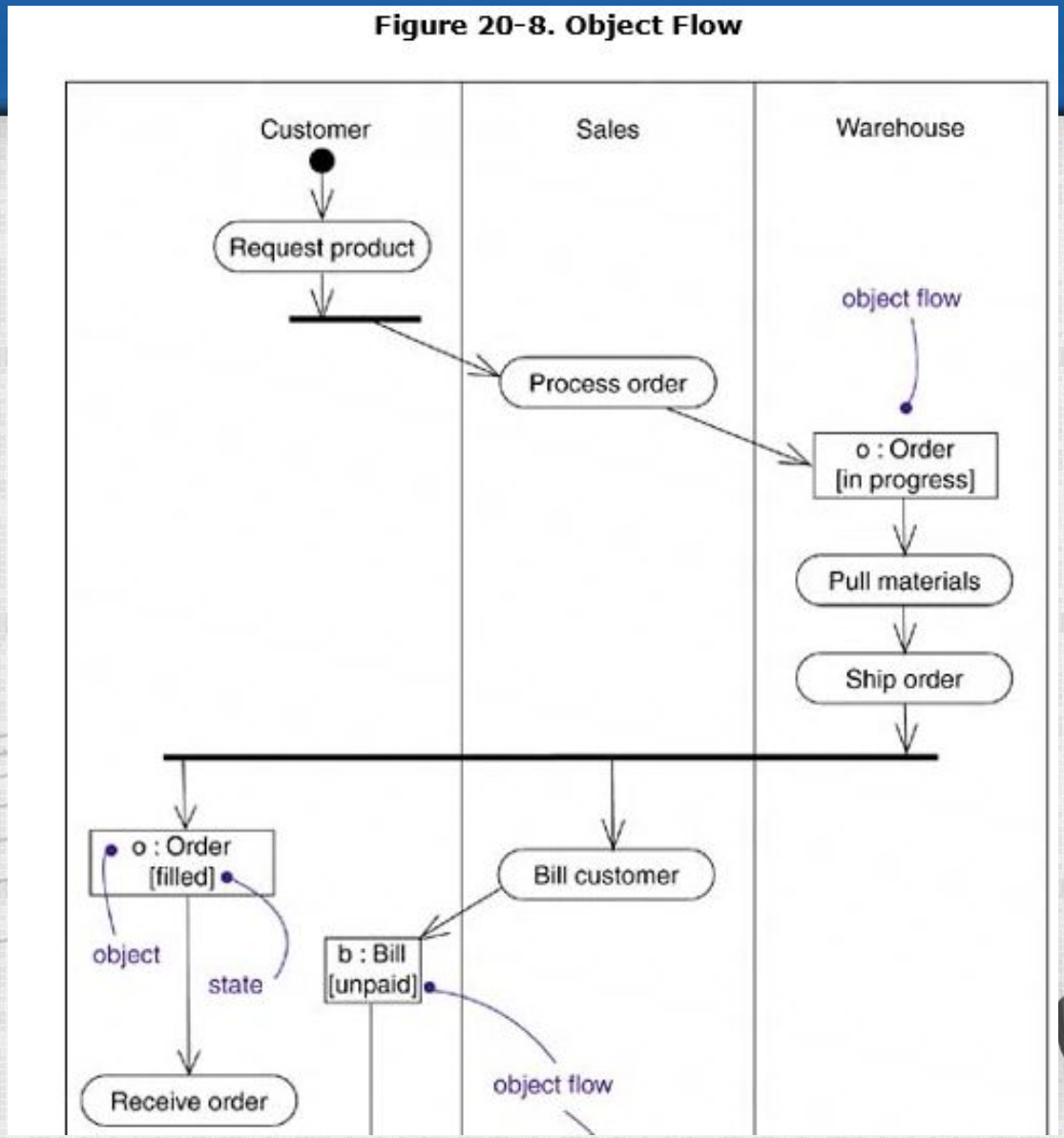



initiation
Select site
Control flow
actions
Commission architect
Develop plan
Bid plan
sequential branch
[not accepted]
[else]
concurrent fork
Do site work
Do trade work
concurrent join
object flow
Finish construction
CertificateOfOccupancy
[completed]
completion
concurrent flows

Business Flow - Process Order Example

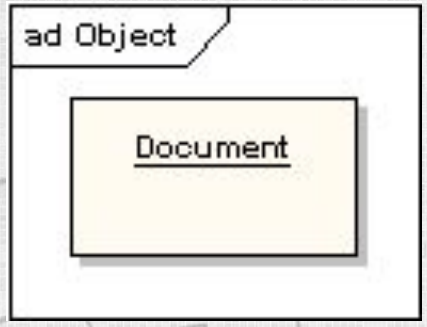


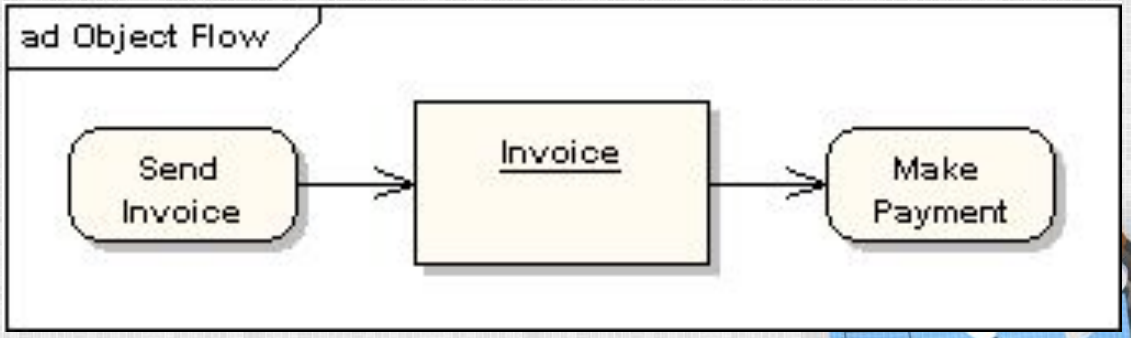
Swimlanes



Object and Object Flow

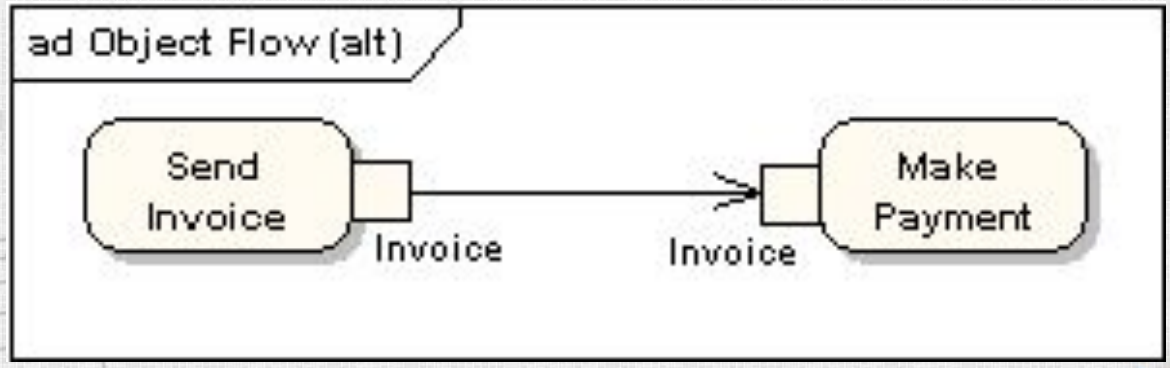
- An object flow is a path along which objects can pass. An object is shown as a rectangle
- An object flow is shown as a connector with an arrowhead denoting the direction the object is being passed.





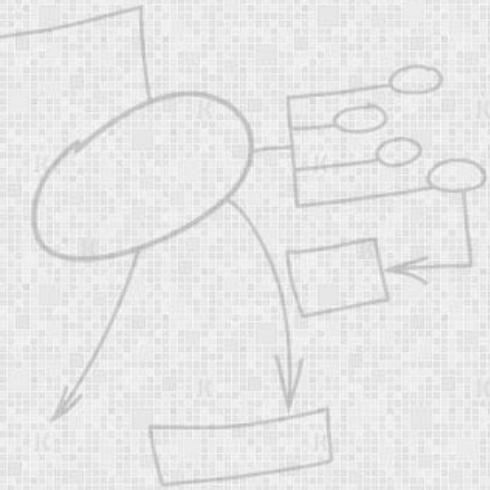
Input and Output Pin

- A shorthand notation for the above diagram would be to use input and output pins



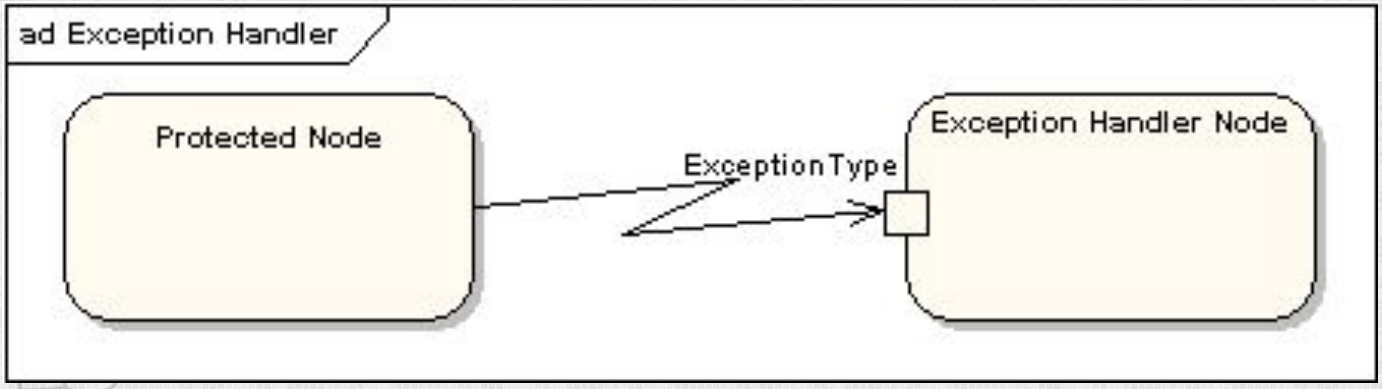
Data Store

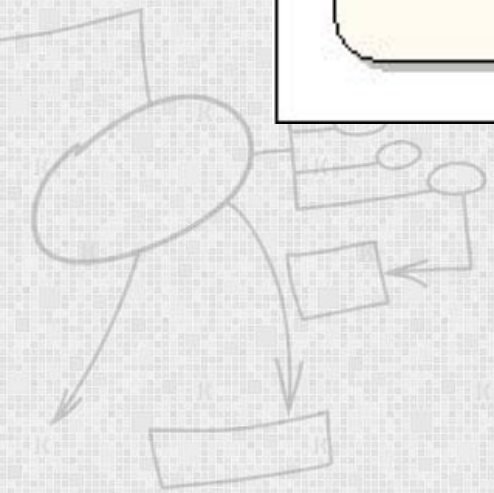
- A data store is shown as an object with the «datastore» keyword



Exception Handling

- Exception Handlers can be modeled on activity diagrams as in the example below





Interruptible Activity Region

- An interruptible activity region surrounds a group of actions that can be interrupted. In the very simple example below, the Process Order action will execute until completion, when it will pass control to the Close Order action, unless a Cancel Request interrupt is received which will pass control to the Cancel Order action